



تمرین یک

برنامه‌سازی پیشرفته
دانشکده علوم مهندسی

طراح تمرین: ایلیا افضلی

بهار ۱۴۰۵



۱. مقدمه و هدف تمرین

در این تمرین با دو اصل بنیادین برنامه‌نویسی شیء‌گرا (OOP) آشنا خواهید شد: کپسوله‌سازی (Encapsulation) و طراحی کلاس‌محور (Class-based Design). کپسوله‌سازی به معنای پنهان‌سازی جزئیات درونی یک موجودیت و ارائه‌ی رابطی کنترل‌شده به دنیای بیرون است؛ در نتیجه، داده و رفتار در کنار هم در یک کلاس قرار می‌گیرند و تغییر پیاده‌سازی داخلی بدون اثرگذاری بر سایر اجزا ممکن می‌شود.

در این تمرین یک سیستم مدیریت وظایف (Todo List) طراحی می‌کنید که از مفهوم پیش‌نیاز (Prerequisite) پشتیبانی می‌کند. هر وظیفه می‌تواند به تعدادی وظیفه‌ی دیگر وابسته باشد و تنها زمانی قابل انجام است که تمام پیش‌نیازهایش به پایان رسیده باشند. این الگو در سیستم‌های مدیریت پروژه، برنامه‌ریزی درسی و زنجیره‌های تولید به وفور به کار می‌رود.

۲. صورت مسئله

یک سیستم مدیریت وظایف تعاملی پیاده‌سازی کنید که دستورات کاربر را خطبه‌خط از ورودی استاندارد دریافت کرده و عملیات متناظر را اجرا می‌کند. هر وظیفه دارای یک شناسه‌ی عددی یکتا، یک عنوان متنی تک‌کلمه‌ای، و یک وضعیت (انجام‌شده یا انجام‌نشده) است. هر وظیفه می‌تواند به تعدادی وظیفه‌ی دیگر به عنوان پیش‌نیاز وابسته باشد.

برنامه با دستور `NEW_LIST` راه‌اندازی می‌شود و تا دریافت دستور `EXIT` به پردازش دستورات ادامه می‌دهد. قالب ورودی همیشه از نظر نحوی صحیح است، اما ممکن است خطاهای منطقی داشته باشد که برنامه باید آن‌ها را مدیریت کند.

۳. الزامات پیاده‌سازی

- کپسوله‌سازی: تمام فیلدهای داده‌ی کلاس‌ها باید `private` باشند و دسترسی به آن‌ها صرفاً از طریق متدهای `public` انجام شود. استفاده از `struct` به جای `class` مجاز نیست.
- طراحی کلاس‌محور: منطق اصلی برنامه باید در کلاس‌های مناسب پیاده‌سازی شود. تابع `main` باید کوتاه بوده و صرفاً مسئول راه‌اندازی و حلقه‌ی دریافت دستور باشد.
- مدیریت پیش‌نیازها: پیش از علامت‌گذاری وظیفه‌ای به عنوان انجام‌شده، تمام پیش‌نیازهای آن باید بررسی شوند. در صورت ناتمام بودن حتی یکی، عملیات با پیام خطای مناسب رد می‌شود.

توجه ⚠

خروجی برنامه باید دقیقاً با فرمت تعریف‌شده در بخش دستورات مطابقت داشته باشد. فاصله‌ها، نقطه‌گذاری‌ها و حروف بزرگ و کوچک همگی در داوری خودکار بررسی می‌شوند.



۴. شرح دستورات

ایجاد لیست جدید

NEW_LIST

این دستور یک لیست وظایف جدید با نام دلخواه ایجاد می‌کند و تمام وضعیت قبلی سیستم را پاک می‌نماید. تضمین شده است که قبل از هر دستور دیگری حداقل یک بار فراخوانی می‌شود. در صورت فراخوانی مجدد، لیست قبلی و تمام وظایف آن حذف شده و لیست جدیدی آغاز می‌شود.

📩 ساختار ورودی:

NEW_LIST <name>

• <name> – نام لیست: یک رشته تک‌کلمه‌ای (بدون فاصله)

📩 ساختار خروجی:

موفق:

```
List '<name>' created.
```

⚠ خطاهای منطقی احتمالی

این دستور در مشخصات تمرین خطایی ندارد؛ فراخوانی مجدد لیست قبلی را بازنویسی می‌کند و خروجی موفقیت چاپ می‌شود.

📌 نمونه

ورودی:

NEW_LIST MyProject

خروجی:

```
List 'MyProject' created.
```



افزودن وظیفه بدون پیش‌نیاز

ADD

یک وظیفه جدید با شناسه عددی یکتا و عنوان مشخص به لیست جاری اضافه می‌کند. این وظیفه هیچ پیش‌نیازی ندارد و بلافاصله در وضعیت آماده (ready) قرار دارد.

ساختار ورودی:

ADD <id> <title>

- <id> – شناسه یکتای وظیفه: عدد صحیح مثبت
- <title> – عنوان وظیفه: یک کلمه تک‌بخشی (بدون فاصله)

ساختار خروجی:

موفق:

Task <id> added.

⚠ خطاهای منطقی احتمالی

- شناسه تکراری: اگر وظیفه‌ای با همین <id> قبلاً وجود داشته باشد.
خروجی: Error: task <id> already exists.

📌 نمونه

ورودی:

ADD 1 Design

خروجی:

Task 1 added.



افزودن وظیفه با پیش‌نیاز

ADD_DEP

یک وظیفه جدید با یک یا چند پیش‌نیاز به لیست جاری اضافه می‌کند. پیش‌نیازها شناسه‌های وظایفی هستند که باید قبل از این وظیفه تکمیل شوند. تا زمانی که تمام پیش‌نیازها به وضعیت *done* نرسیده باشند، این وظیفه قابل انجام نیست.

ساختار ورودی:

```
ADD_DEP <id> <title> <dep1> [dep2 dep3 ...]
```

- `<id>` – شناسه یکتای وظیفه جدید: عدد صحیح مثبت
- `<title>` – عنوان وظیفه جدید: یک کلمه تک‌بخشی
- `<dep1> [dep2 ...]` – شناسه‌ی یک یا چند وظیفه‌ی پیش‌نیاز (حداقل یکی الزامی)

ساختار خروجی:

موفق:

```
Task <id> added with <N> deps.
```

`<N>` = تعداد پیش‌نیازهای ارائه‌شده

⚠ خطاهای منطقی احتمالی

- شناسه تکراری: اگر وظیفه‌ای با همین `<id>` از پیش وجود داشته باشد.
خروجی: `Error: task <id> already exists.`
- پیش‌نیاز ناموجود: اگر یکی از `<depN>` ها در لیست وظایف یافت نشود.
خروجی: `Error: dependency <depN> not found.`

🚀 نمونه

ورودی:

```
ADD_DEP 4 Testing 2 3
```

خروجی (موفق – وظایف ۲ و ۳ وجود دارند):

```
Task 4 added with 2 deps.
```

ورودی (خطا):

```
ADD_DEP 5 Deploy 99
```

خروجی (پیش‌نیاز ناموجود):

```
Error: dependency 99 not found.
```



تکمیل وظیفه

DONE

وظیفه‌ای با شناسه مشخص را به وضعیت «انجام‌شده» (*done*) تغییر می‌دهد. این عملیات تنها زمانی موفق است که اولاً وظیفه در لیست موجود باشد، ثانیاً تمام پیش‌نیازهای آن قبلاً به وضعیت *done* رسیده باشند.

ساختار ورودی:

DONE <id>

• شناسه وظیفه‌ای که باید تکمیل شود <id> –

ساختار خروجی:

موفق:

Task <id> marked as done.

⚠ خطاهای منطقی احتمالی

• شناسه ناموجود: هیچ وظیفه‌ای با این <id> در لیست وجود ندارد.

خروجی: Error: task <id> not found.

• پیش‌نیاز ناتمام: یک یا چند پیش‌نیاز هنوز به وضعیت *done* نرسیده‌اند.

خروجی: Cannot complete task <id>: prerequisites not met.

📌 نمونه

ورودی (موفق – پیش‌نیازها تکمیل شده):

DONE 1

Task 1 marked as done.

ورودی (خطا – پیش‌نیازها تکمیل نشده):

DONE 4

Cannot complete task 4: prerequisites not met.



نمایش وضعیت وظیفه STATUS

اطلاعات کامل یک وظیفه شامل شناسه، عنوان، وضعیت فعلی و لیست کامل پیش‌نیازها را نمایش می‌دهد. در صورتی که وظیفه پیش‌نیاز نداشته باشد، سطر Dependencies حذف می‌شود.

ساختار ورودی:

STATUS <id>

• شناسه وظیفه موردنظر <id> –

ساختار خروجی:

موفق:

```
Task <id>: <title>
Status: pending | done
Dependencies: <dep1> (<title1>), <dep2> (<title2>), ...
```

خطاهای منطقی احتمالی

• شناسه ناموجود: هیچ وظیفه‌ای با این <id> در لیست وجود ندارد.

خروجی: Error: task <id> not found.

!! توجه

- تمامی پیش‌نیازها باید به ترتیب صعودی شناسه چاپ شوند.
- پس از آخرین پیش‌نیاز کاما (,) قرار نگیرد.

نمونه

ورودی:

STATUS 4

خروجی:

```
Task 4: Testing
Status: pending
Dependencies: 2 (Backend), 3 (Frontend)
```



چاپ تمام وظایف

PRINT_ALL

تمام وظایف موجود در لیست جاری را به ترتیب صعودی شناسه (`id`) نمایش می‌دهد. وضعیت هر وظیفه به صورت `[done]` یا `[pending]` در ابتدای هر سطر نشان داده می‌شود. این دستور آرگومانی دریافت نمی‌کند.

ساختار ورودی:

PRINT_ALL

هیچ آرگومانی ندارد.

ساختار خروجی:

یک سطر به ازای هر وظیفه:

```
[done]      <id>: <title>
[pending] <id>: <title>
```

توجه: فاصله‌گذاری بین برچسب وضعیت و شناسه باید دقیقاً مطابق فرمت باشد (چهار فاصله بعد از `[done]` و یک فاصله بعد از `[pending]`).

خطاهای منطقی احتمالی

- لیست خالی: اگر هیچ وظیفه‌ای در لیست وجود نداشته باشد، هیچ خروجی‌ای چاپ نمی‌شود (خط خالی).
- ترتیب نادرست: وظایف باید بر اساس `id` به صورت صعودی مرتب شوند، نه بر اساس ترتیب افزوده‌شدن.

نمونه

ورودی:

PRINT_ALL

خروجی (وظایف ۱ و ۲ تکمیل شده، ۳ در انتظار):

```
[done]      1: Design
[done]      2: Backend
[pending]   3: Frontend
```




وظایف در انتظار

PENDING

فهرست تمام وظایفی که هنوز به وضعیت *done* نرسیده‌اند را در یک سطر نمایش می‌دهد. هر وظیفه با شناسه و عنوان خود نشان داده می‌شود. این دستور هیچ آرگومانی ندارد.

ساختار ورودی:

PENDING

هیچ آرگومانی ندارد.

ساختار خروجی:

موفق:

Pending tasks: <id1> (<title1>), <id2> (<title2>), ...

خطاهای منطقی احتمالی

- همه تکمیل شده‌اند: اگر تمام وظایف *done* باشند یا لیست خالی باشد، هیچ خروجی‌ای چاپ نمی‌شود (خط خالی).

!! توجه

- تمامی وظایف *pending* باید به ترتیب صعودی شناسه چاپ شوند.
- پس از آخرین وظیفه کاما (,) قرار نگیرد.

نمونه

ورودی:

PENDING

خروجی (وظیفه ۵ هنوز انجام نشده):

Pending tasks: 5 (Deploy)



وظایف آماده انجام

READY

فهرست وظایفی را نمایش می‌دهد که هنوز انجام نشده‌اند اما تمام پیش‌نیازهایشان به وضعیت *done* رسیده است. وظایف بدون پیش‌نیاز که هنوز *pending* هستند نیز در این فهرست قرار می‌گیرند. خروجی فقط شناسه‌های این وظایف است.

ساختار ورودی:

READY

هیچ آرگومانی ندارد.

ساختار خروجی:

موفق:

Ready tasks: <id1> <id2> ...

توجه: برخلاف **PENDING**، در خروجی این دستور عنوان وظایف ذکر نمی‌شود – فقط شناسه‌ها با فاصله جدا می‌شوند.

⚠ خطاهای منطقی احتمالی

- هیچ وظیفه آماده‌ای نیست: اگر همه وظایف باقیمانده دارای پیش‌نیاز ناتمام باشند، هیچ خروجی‌ای چاپ نمی‌شود (خط خالی).
- اشتباه در منطق **ready**: وظیفه‌ای که خودش *done* است نباید در این لیست ظاهر شود، حتی اگر پیش‌نیازهایش هم *done* باشند.

!! توجه

- تمامی وظایف **ready** باید به ترتیب صعودی شناسه چاپ شوند.
- پس از آخرین وظیفه کاما (,) قرار نگیرد.

📌 نمونه

ورودی:

READY

خروجی (وظایف ۲ و ۳ بدون پیش‌نیاز و هنوز *pending*):

Ready tasks: 2 3



حذف وظیفه

REMOVE

وظیفه‌ای با شناسه مشخص را از لیست حذف می‌کند. این عملیات تنها زمانی مجاز است که هیچ وظیفه دیگری در لیست به این وظیفه به عنوان پیش‌نیاز وابسته نباشد. در صورت وجود وابسته، حذف رد می‌شود.

ساختار ورودی:

REMOVE <id>

• <id> – شناسه وظیفه‌ای که باید حذف شود

ساختار خروجی:

موفق:

Task <id> removed.

⚠ خطاهای منطقی احتمالی

- شناسه ناموجود: هیچ وظیفه‌ای با این <id> در لیست وجود ندارد.
خروجی: Error: task <id> not found.
- وجود وابسته: یک یا چند وظیفه دیگر این وظیفه را به عنوان پیش‌نیاز خود ثبت کرده‌اند و حذف آن زنجیره وابستگی را خراب می‌کند.
خروجی: Cannot remove task <id>: other tasks depend on it.

📌 نمونه

ورودی (موفق – هیچ وظیفه‌ای به ۱ وابسته نیست):

REMOVE 1

Task 1 removed.

ورودی (خطا – وظیفه ۴ به ۲ وابسته است):

REMOVE 2

Cannot remove task 2: other tasks depend on it.



پایان برنامه

EXIT

با دریافت این دستور، برنامه متوقف شده و دیگر هیچ ورودی ای خوانده نمی شود. تمام منابع آزاد می شوند و اجرا خاتمه می یابد.

ساختار ورودی:

EXIT

هیچ آرگومانی ندارد.

ساختار خروجی:

خروجی:

(بدون خروجی) -

⚠ خطاهای منطقی احتمالی

این دستور خودش خطایی تولید نمی کند. تنها نکته اینکه اگر دستوراتی بعد از EXIT در ورودی وجود داشته باشند، باید نادیده گرفته شوند.

📌 نمونه

ورودی:

EXIT

خروجی:

(برنامه پایان می یابد)



۵. مثال

ورودی نمونه:

```
NEW_LIST MyProject
ADD 1 Design
ADD 2 Backend
ADD 3 Frontend
ADD_DEP 4 Testing 2 3
ADD_DEP 5 Deploy 4
DONE 1
STATUS 4
DONE 4
DONE 2
DONE 3
DONE 4
PRINT_ALL
PENDING
READY
REMOVE 1
REMOVE 2
EXIT
```

خروجی نمونه:

```
List 'MyProject' created.
Task 1 added.
Task 2 added.
Task 3 added.
Task 4 added with 2 deps.
Task 5 added with 1 deps.
Task 1 marked as done.
Task 4: Testing
Status: pending
Dependencies: 2 (Backend), 3 (Frontend)
Cannot complete task 4: prerequisites not met.
Task 2 marked as done.
Task 3 marked as done.
Task 4 marked as done.
[done] 1: Design
[done] 2: Backend
[done] 3: Frontend
[done] 4: Testing
[pending] 5: Deploy
Pending tasks: 5 (Deploy)
Ready tasks: 5
Task 1 removed.
Cannot remove task 2: other tasks depend on it.
```



۶. ساختار کد پیشنهادی

ساختار زیر صرفاً پیشنهادی است و می‌توانید طراحی متفاوتی داشته باشید:

```
// Task Class
class Task {
private:
    int id;
    string title;
    bool done;
    vector<int> dependencies;
public:
    Task(int id, string title);
    void addDependency(int depId);
    bool canComplete(/* ... */);
    void markDone();
    // getters ...
};
```

```
// TodoList Class
class TodoList {
private:
    string name;
    vector<Task> tasks;
    Task* findTask(int id);
public:
    TodoList(string name);
    void addTask(int id, string title);
    void addTaskWithDeps(int id, string title,
                        vector<int> deps);
    void markDone(int id);
    void printAll();
    void printPending();
    void printReady();
    void removeTask(int id);
};
```